

Introduction to Machine Learning

Introduction

- Machine learning models are computer programs that are used to recognize patterns in data or make predictions.
 - *Machine learning is a subset of artificial intelligence (AI) that focuses on building systems that learn from data and make decisions or predictions without being explicitly programmed for every task.*
 - *Instead of writing specific rules or algorithms for every task, machine learning models are trained on data and improve their performance based on experience.*

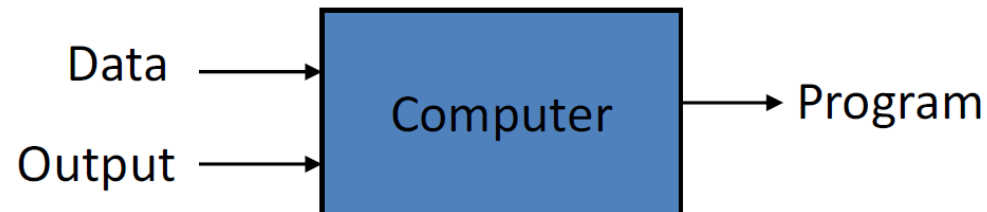
Machine Learning (ML)

“A computer program is said to learn from experience (E) with some class of tasks (T) and a performance measure (P) if its performance at tasks in T as measured by P improves with E”

Traditional Programming



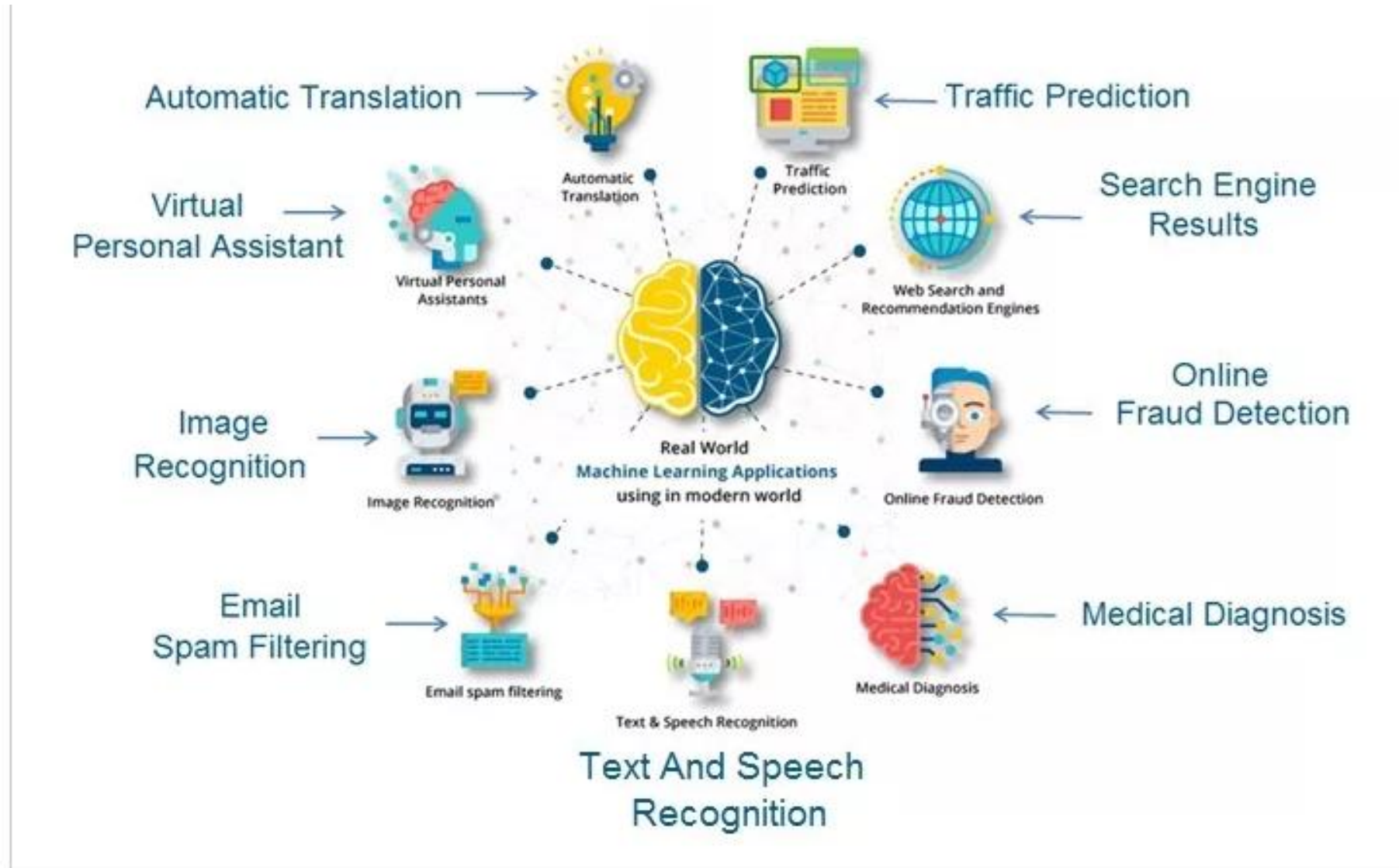
Machine Learning



Why is ML needed?

- Traditional programming is rule based
 - Programs implement rules, computers follow them exactly.
 - Problem: Many real world problems do not have clear rules. E.g. What rules can be defined to recognize a face?
- Explosion of data
 - Massive amount of data is generated – social media, online transactions, sensors and IoT devices.
 - Traditional programming is not capable of processing and extracting insights from the huge amount of data.
- ML systems improve with experience
 - Traditional programs remain the same unless reprogrammed.
 - ML algorithms improve, adapt and update themselves with experience.

Applications

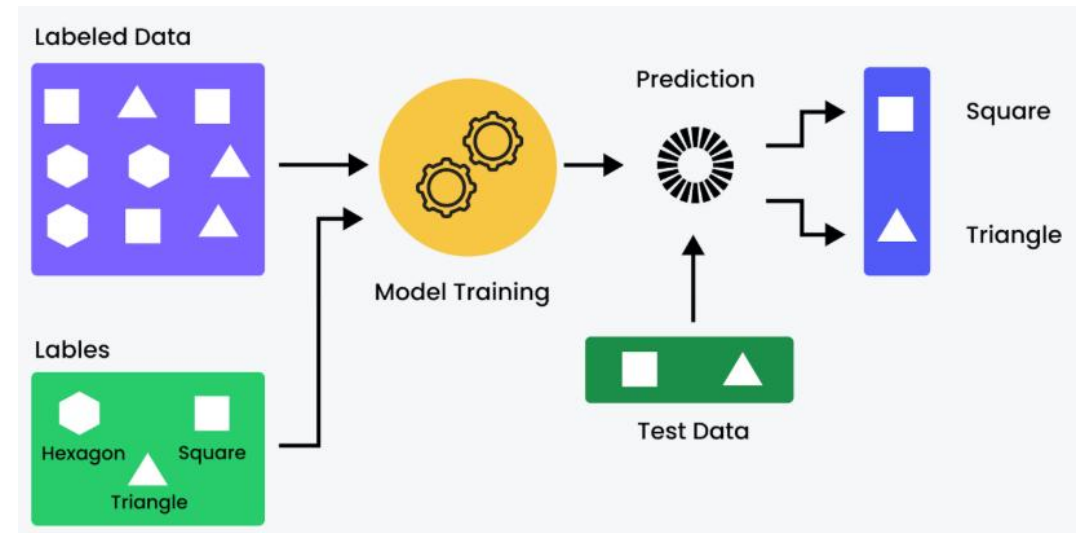
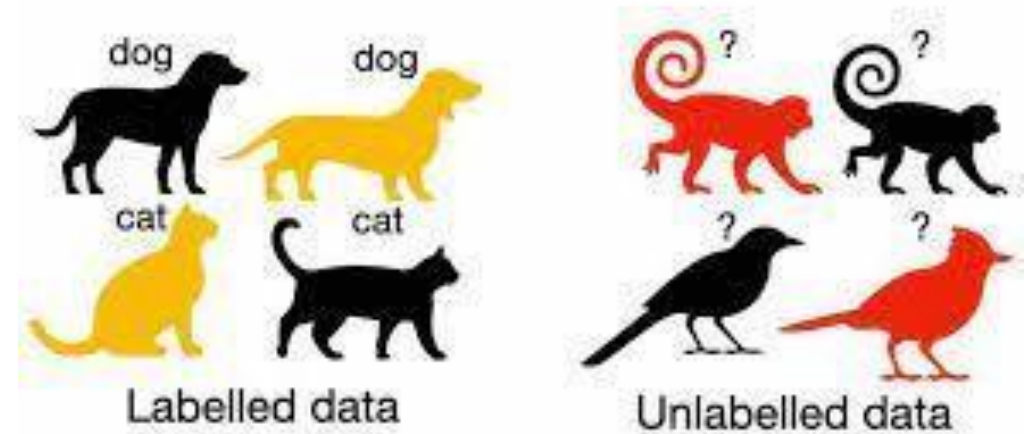


Types

- Supervised Learning
- Unsupervised Learning
- Semi supervised learning
- Reinforcement Learning

Supervised Learning

- Training data is labelled containing both descriptions(input) and the target(output).
 - *training data (input, target)*
 - *input – description of a face, **area of a house***
 - *target – happy, **price of the house***
- The algorithm (model) tries to determine the relationship between the input and the output.
- This knowledge is then used to predict the output for previously unseen inputs.



Supervised Learning: Types

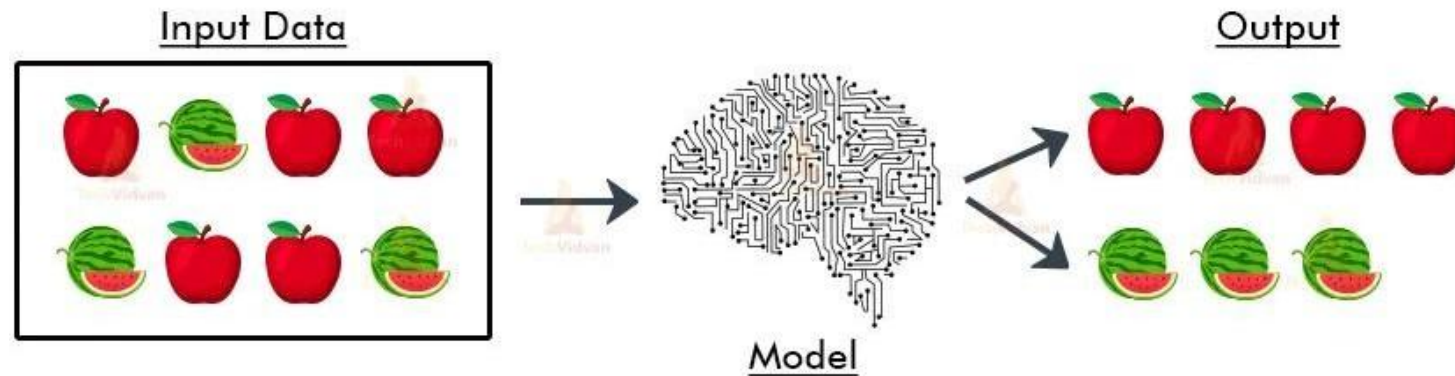
- Regression
 - the model predicts a **continuous output**.
 - E.g. predicting house prices based on features like size, number of bedrooms, and location.
- Classification
 - the model assigns data points to **predefined classes**.
 - E.g., classifying emails as spam or not spam.

Algorithms: Linear regression, decision trees, support vector machine

Unsupervised Learning

- Algorithm is trained on unlabeled data to discover patterns within the data.
- No predefined target outputs or labels.

Unsupervised Learning in ML



- Example: customer segmentation,

Unsupervised Learning

Top coverage

 **The Indian EXPRESS**

England vs Pakistan 1st Test Day 3 Live Cricket Score
Updates: ENG trail by 167 runs at Lunch

23 minutes ago



 **INDIA TODAY**

You made him carry shoes: Shoaib Akhtar unhappy with former Pakistan captain Sarfaraz Ahmed carrying drinks

4 hours ago



hindustantimes

If he was Virat Kohli everyone would be talking but because it is Babar Azam, no one is: Nasser Hus...

Yesterday



 **ZEE NEWS**

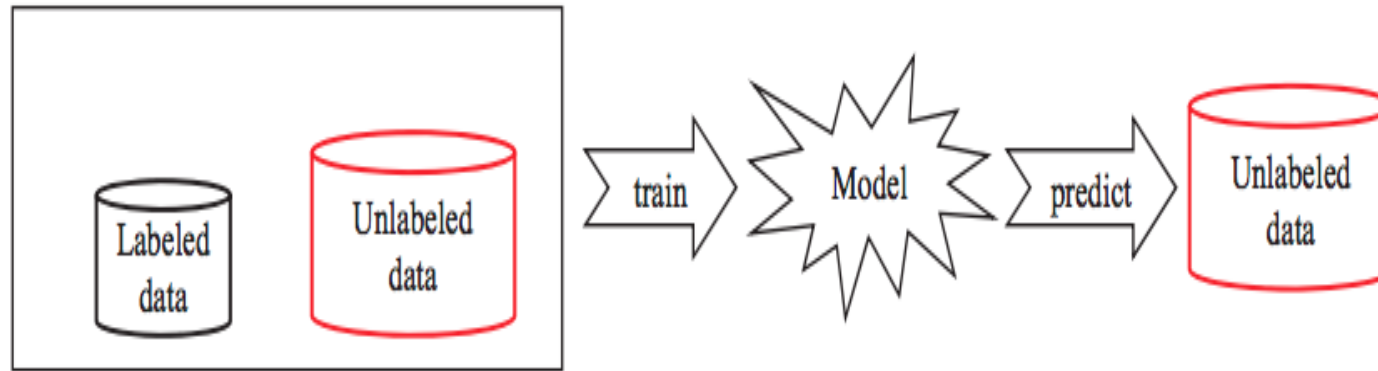
1st Test Day 3 Live Updates: England reach 159/5 against Pakistan at Lunch

58 minutes ago



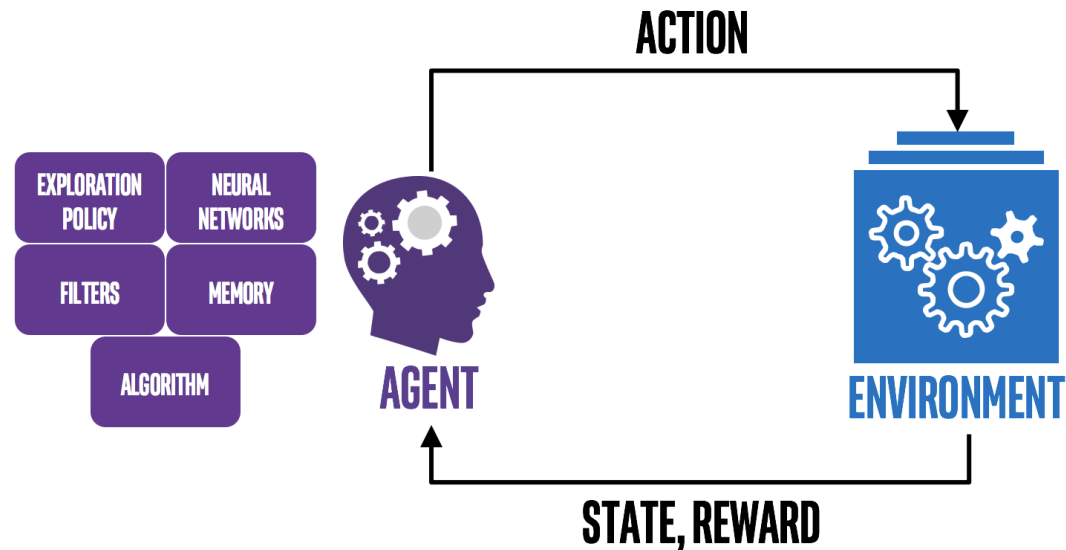
Semi-Supervised Learning

- Combines a small amount of labeled data with a large amount of unlabeled data during training.



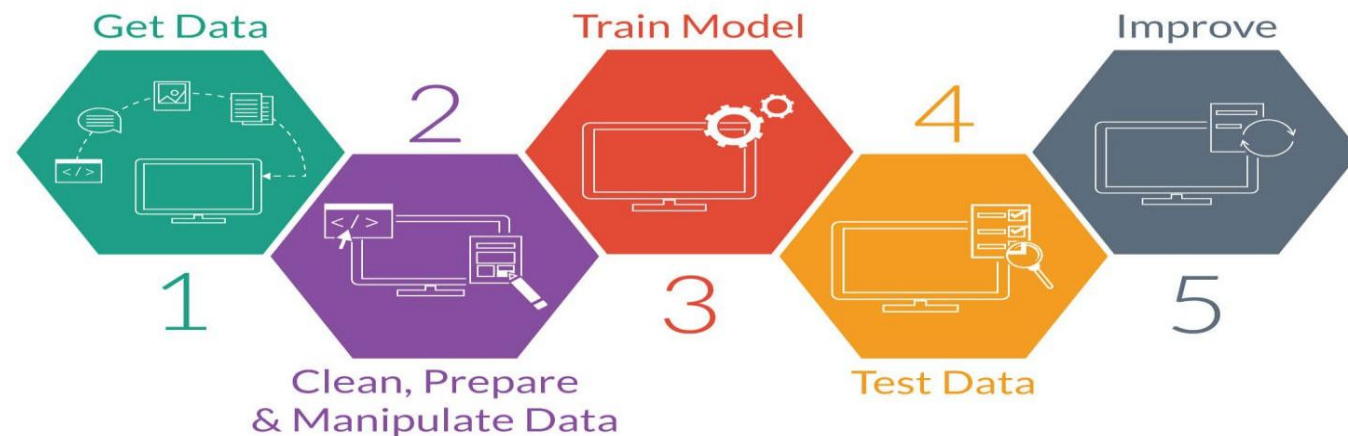
Reinforcement Learning

- It is the training of machine learning models to make a sequence of decisions.
- The computer employs trial and error to come up with a solution to the problem.
- The agent gets either rewards or penalises for the actions it performs.



Basic ML workflow

- Data collection
- Data preprocessing
- Feature selection
- Model training
- Evaluation
- Prediction / Deployment



Data Preprocessing

- Cleaning, transforming and performing other such operations on the data so that it can be effectively used by ML algorithms.

Why is it necessary?

- Real-world data is often incomplete, noisy, and inconsistent.
- Proper/processed data improves model accuracy, reduces training time, prevents misleading patterns, ensures fair and unbiased learning.

Steps in Data Preprocessing

- Data Cleaning
- Data Integration
- Data Transformation
- Data Encoding
- Feature Selection
- Feature Extraction
- Handling Imbalanced Data
- Data Splitting

Data Cleaning

- Handling missing values (mean, median, mode, deletion).
- Removing duplicate records
- Correcting inconsistent data

Data Integration

- Combines data from multiple sources into a single dataset.

Data Transformation

- Converts data into a suitable format.
- E.g scaling values to range $[0, 1]$

Data Encoding

- Converts categorical data into numerical form.
- Example: Label encoding, One-Hot encoding

Feature selection

- Selecting the most relevant features.

Feature Extraction

- Creates new features from existing data.

Handling Imbalanced Data

- Ensuring that the dataset has a fair representation of the population.

Data Splitting

- Divides dataset into: Training set, Validation set, Test set

Example using a sample dataset

ID	Area (sqft)	Bedrooms	Location	Price (₹ Lakhs)
1	1200	2	Urban	50
2	1500	3	Urban	65
3	NaN	3	Rural	40
4	800	2	Rural	NaN
5	1200	2	Urban	50

Issues in the dataset

- *Missing values (NaN)*
- *Categorical data (Location)*
- *Duplicate record (row 1 and row 5)*
- *Different value scales*

Example using a sample dataset

Handling missing values (NaN)

- Replace missing Area with mean
- Replace missing Price with mean
- Mean Area = $(1200 + 1500 + 800 + 1200) / 4 = 1175$
- Mean Price = $(50 + 65 + 40 + 50) / 4 = 51.25$

ID	Area (sqft)	Bedrooms	Location	Price (₹ Lakhs)
1	1200	2	Urban	50
2	1500	3	Urban	65
3	NaN	3	Rural	40
4	800	2	Rural	NaN
5	1200	2	Urban	50

Before

ID	Area (sqft)	Bedrooms	Location	Price (₹ Lakhs)
1	1200	2	Urban	50
2	1500	3	Urban	65
3	1175	3	Rural	40
4	800	2	Rural	51.25
5	1200	2	Urban	50

After

Example using a sample dataset

Removing Duplicates

- Row 5 is a duplicate of Row 1 → remove Row 5

ID	Area (sqft)	Bedrooms	Location	Price (₹ Lakhs)
1	1200	2	Urban	50
2	1500	3	Urban	65
3	1175	3	Rural	40
4	800	2	Rural	51.25
5	1200	2	Urban	50

Before

ID	Area (sqft)	Bedrooms	Location	Price (₹ Lakhs)
1	1200	2	Urban	50
2	1500	3	Urban	65
3	1175	3	Rural	40
4	800	2	Rural	51.25

After

Example using a sample dataset

Data Encoding (Categorical → Numerical)

One-Hot Encoding.

	Urban	Rural
Urban	1	0
Rural	0	1

ID	Area (sqft)	Bedrooms	Location	Price (₹ Lakhs)
1	1200	2	Urban	50
2	1500	3	Urban	65
3	1175	3	Rural	40
4	800	2	Rural	51.25

Before

ID	Area (sqft)	Bedrooms	Urban	Rural	Price (₹ Lakhs)
1	1200	2	1	0	50
2	1500	3	1	0	65
3	1175	3	0	1	40
4	800	2	0	1	51.25

After

Example using a sample dataset

Feature Scaling (Normalization)

Normalize Area so large values do not dominate.

Normalization formula: $X_{\text{norm}} = (X - X_{\text{min}}) / (X_{\text{max}} - X_{\text{min}})$

Min Area = 800

Max Area = 1500

For Area = 1200, $X_{\text{norm}} = (1200 - 800) / (1500 - 800) = 0.57$

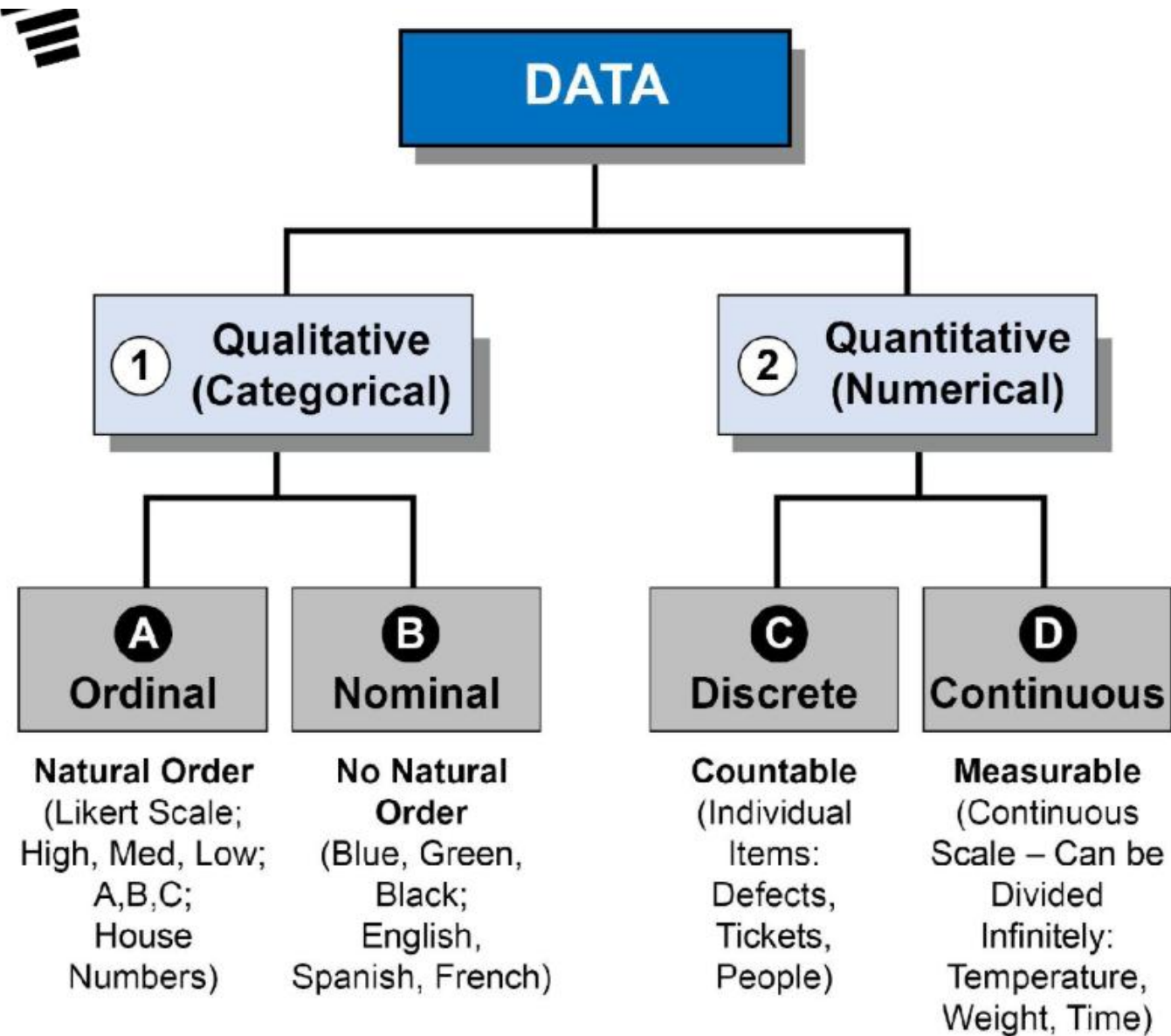
ID	Area (sqft)	Bedrooms	Urban	Rural	Price (₹ Lakhs)
1	1200	2	1	0	50
2	1500	3	1	0	65
3	1175	3	0	1	40
4	800	2	0	1	51.25

Before

ID	Area (sqft)	Bedrooms	Urban	Rural	Price (₹ Lakhs)
1	0.57	2	1	0	50
2	1.00	3	1	0	65
3	0.54	3	0	1	40
4	0.00	2	0	1	51.25

After

Types of data



Continuous data

Interval

- *Equal intervals between values*
- *No true zero (zero does not mean none)*
- *Addition and subtraction are meaningful*
- *E.g. Date, temperature, time*

Ratio

- *Like interval data but has true zero*
- *Addition, subtraction, multiplication and division are meaningful.*
- *E.g. Speed, distance*

Linear Regression

- Linear regression is used for modeling the relationship between a dependent variable (target) and one or more independent variables (predictors or features) by fitting a linear line to the observed data.
- The aim is to find the best fitting line that describes the relationship.
- Types
 - Simple Linear Regression
 - Multiple Linear Regression

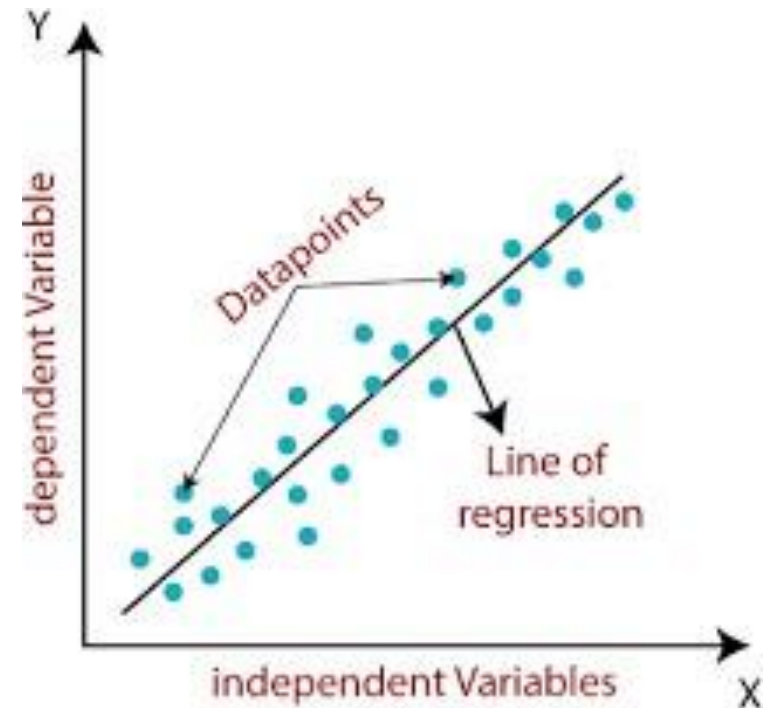


Fig. Simple Linear Regression

Simple Linear Regression

- One independent and one dependent variable.
- Simple Linear regression model is represented by the following equation: -

$$Y = \beta_0 + \beta_1 X$$

where, β_0 – y intercept, β_1 – slope
X – independent variable
Y – dependent variable

- The slope and the intercept of the best fitting line determines the relationship between the variables.

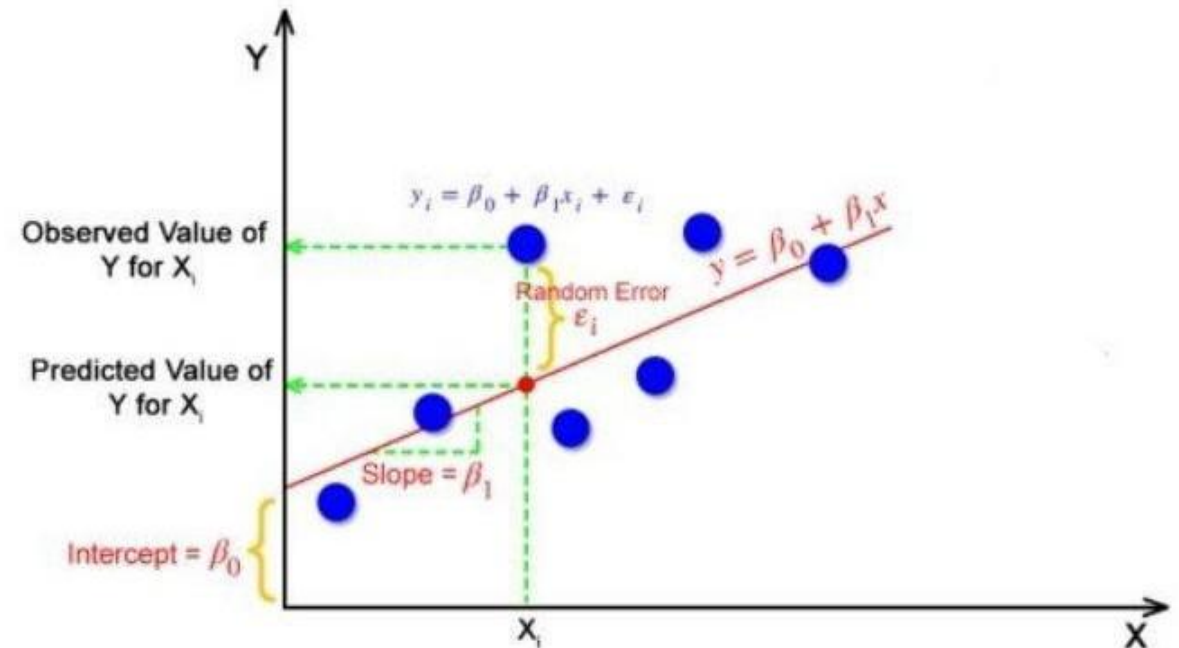
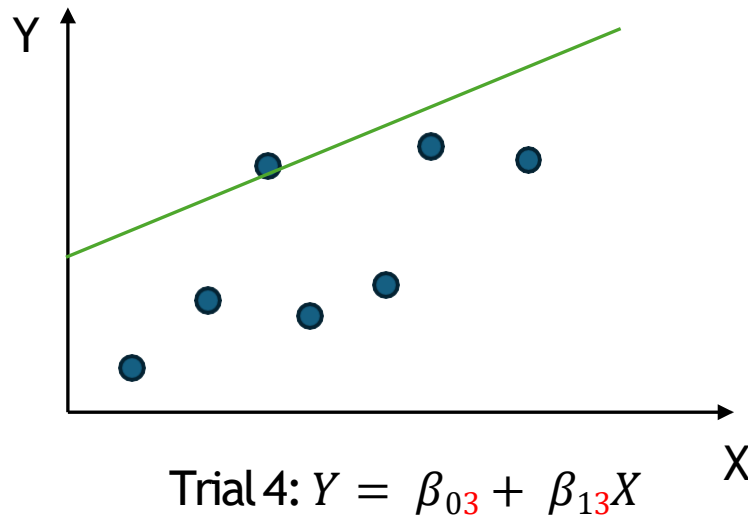
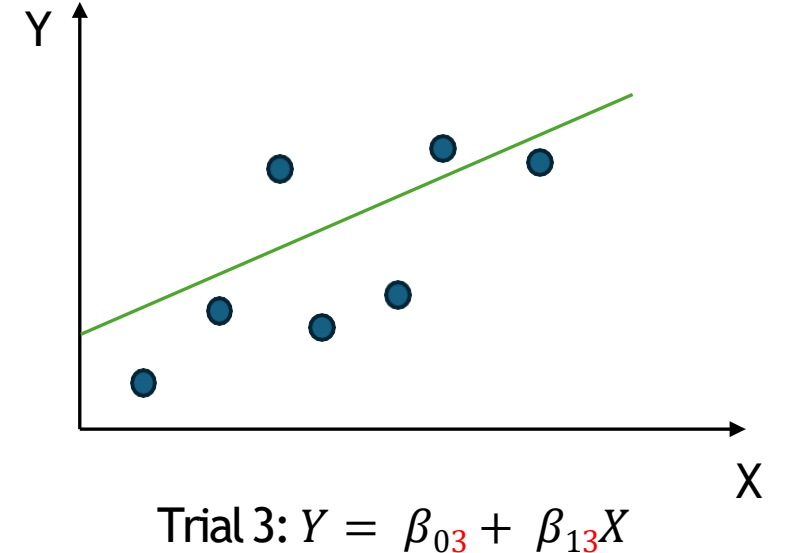
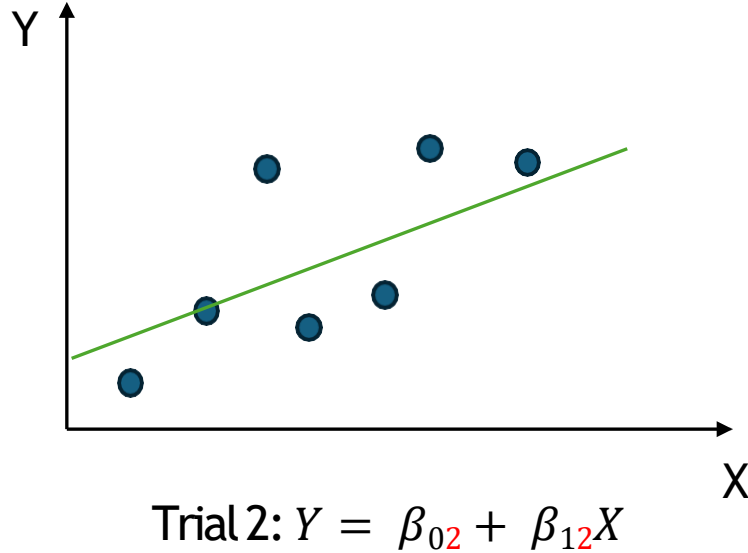
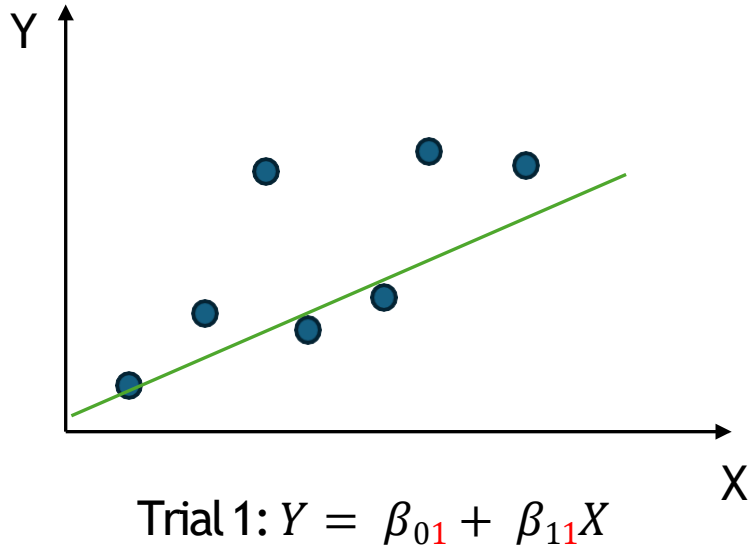


Fig. Simple Linear Regression

Simple Linear Regression: How is the best fitting line found?



- Multiple trials of fitting a line are conducted, each time with a different line
- Error between the actual and predicted value is computed. The line with the least error is the best fitting line.

Simple Linear Regression: Residual Sum of Squares

- Residual = $y_i - y_i'$,
 - where, y_i = actual value, y_i' = predicted value
- Mean Squared Error (MSE) = $\frac{1}{n} \sum (y_i - y_i')^2$
$$= \frac{1}{n} \sum (y_i - (\beta_0 + \beta_1 x_i))^2$$

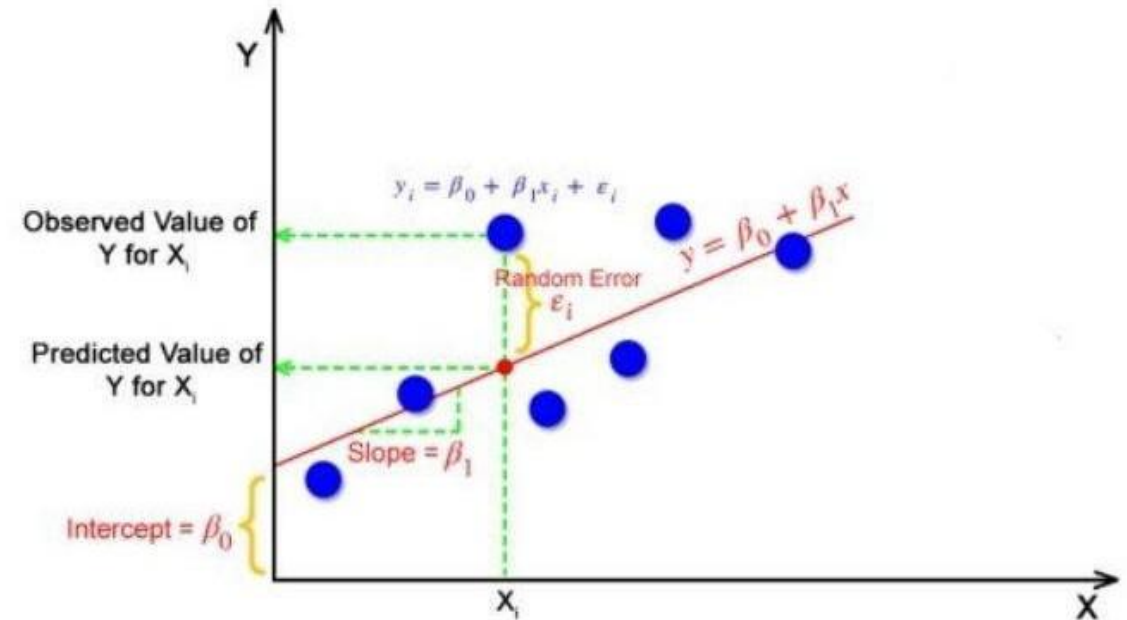


Fig. Simple Linear Regression

Link to video:

https://www.youtube.com/watch?v=owl7zxCqNY0Cab_channel=dataminingincae

Linear Regression: House Price Prediction

Area (sq. ft.)	Price (lakhs)
500	12
650	13.5
720	16.5
750	17
780	17.2
850	19
880	21
1050	24
1200	29
1550	41

Data set

80-20
split

Area (sq. ft.)	Price (lakhs)
500	12
650	13.5
720	16.5
750	17
780	17.2
850	19
880	21
1050	24

Training set

Area (sq. ft.)	Price (lakhs)
1200	29
1550	41

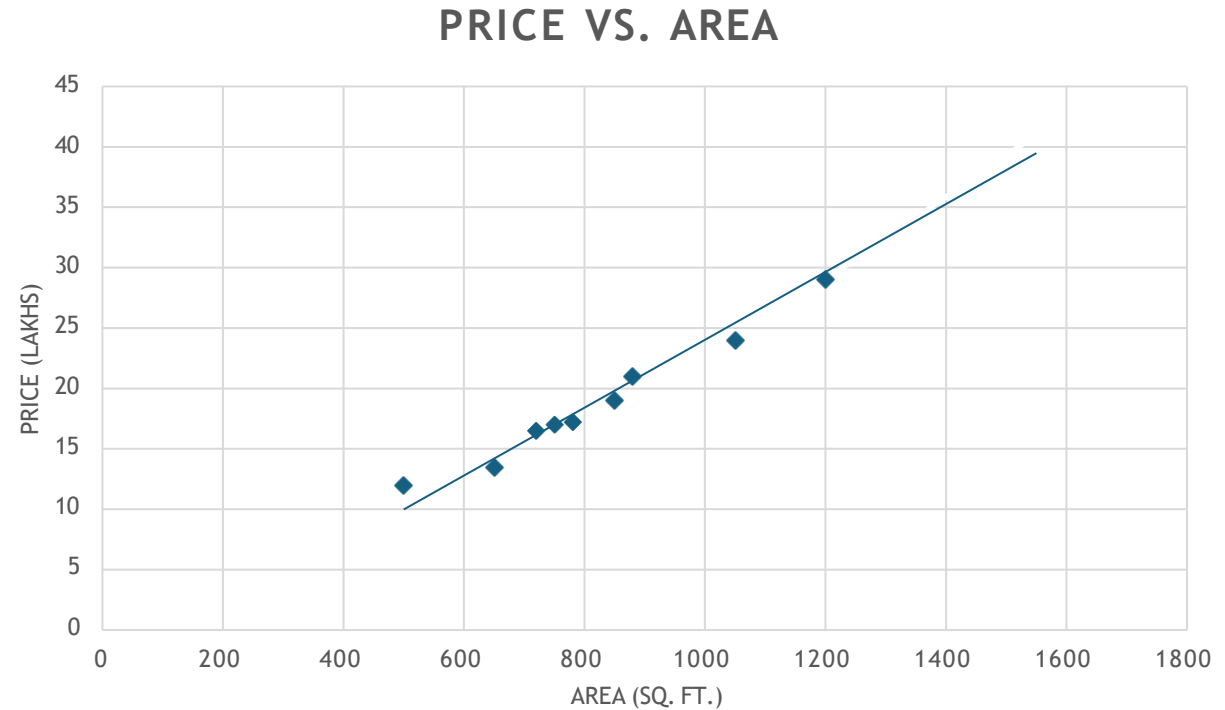
Testing set

- Training set is used for training (finding the best fit line).
- Testing set is used for evaluating the trained model.

Linear Regression: House Price Prediction (Training)

Area (sq. ft.)	Price (lakhs)
500	12
650	13.5
720	16.5
750	17
780	17.2
850	19
880	21
1050	24

Training set



- Best fitting line found by adjusting the slope (β_1) and y intercept (β_0).
- Suppose $\beta_0 = 3$ and $\beta_1 = 6$ are the determined values, then the equation of the best fitting line is $Y = 3 + 6X$

Linear Regression: House Price Prediction (Testing)

Area (sq. ft.)	Price (lakhs)
1200	29
1550	41

Testing set

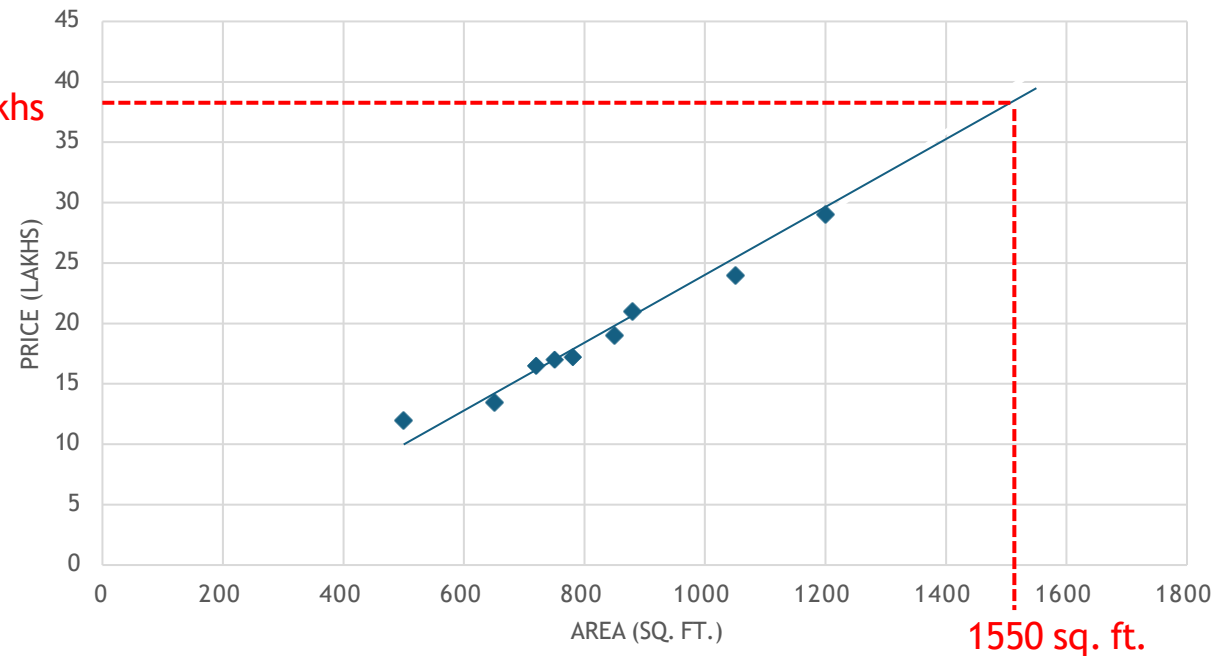
Actual
price

$$Y = 0.242x + 0.51$$

Predicted
price

38 lakhs

PRICE VS. AREA



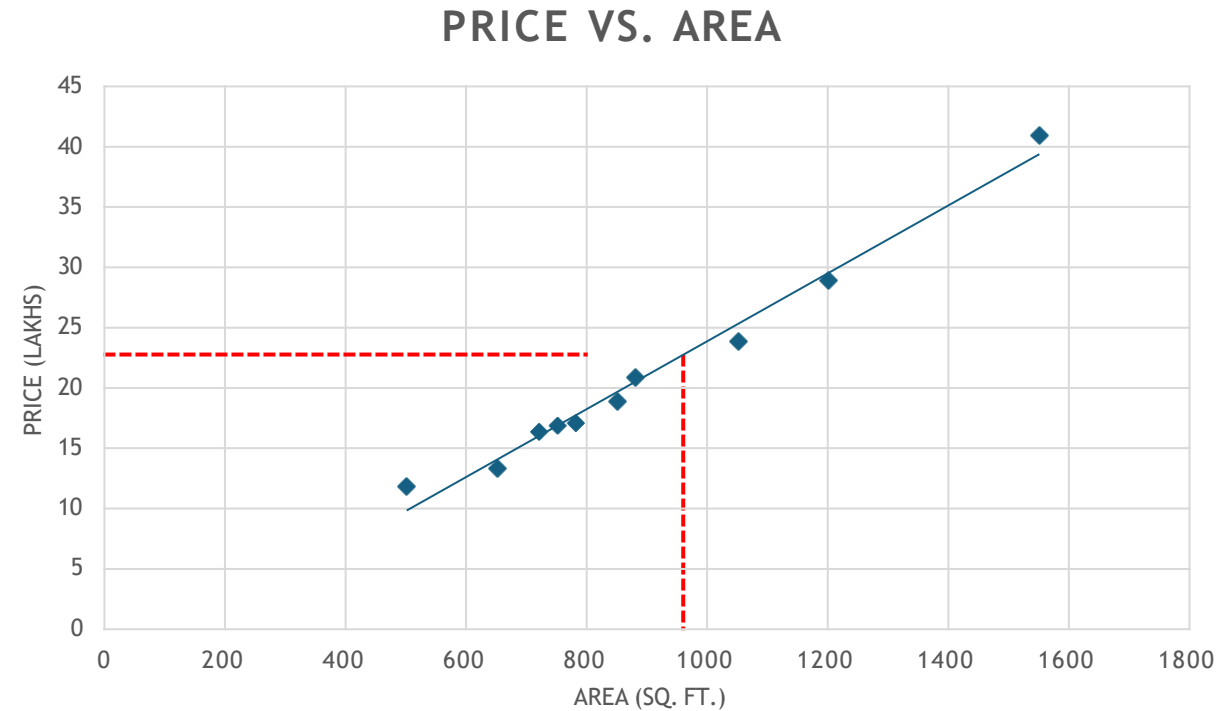
Price for the examples in the testing set predicted using the regression line.

Linear Regression: House Price Prediction

Prediction phase

Area (sq. ft.)	Price (lakhs)
500	12
650	13.5
720	16.5
750	17
780	17.2
850	19
880	21
1050	24
1200	29
1550	41

Data set

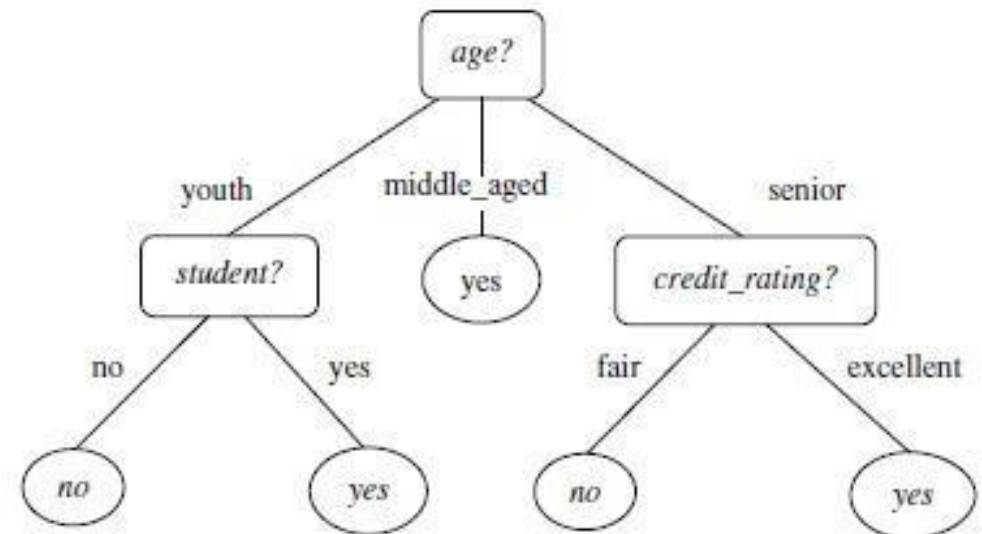


Price of a house with area of 900 sq. ft.?

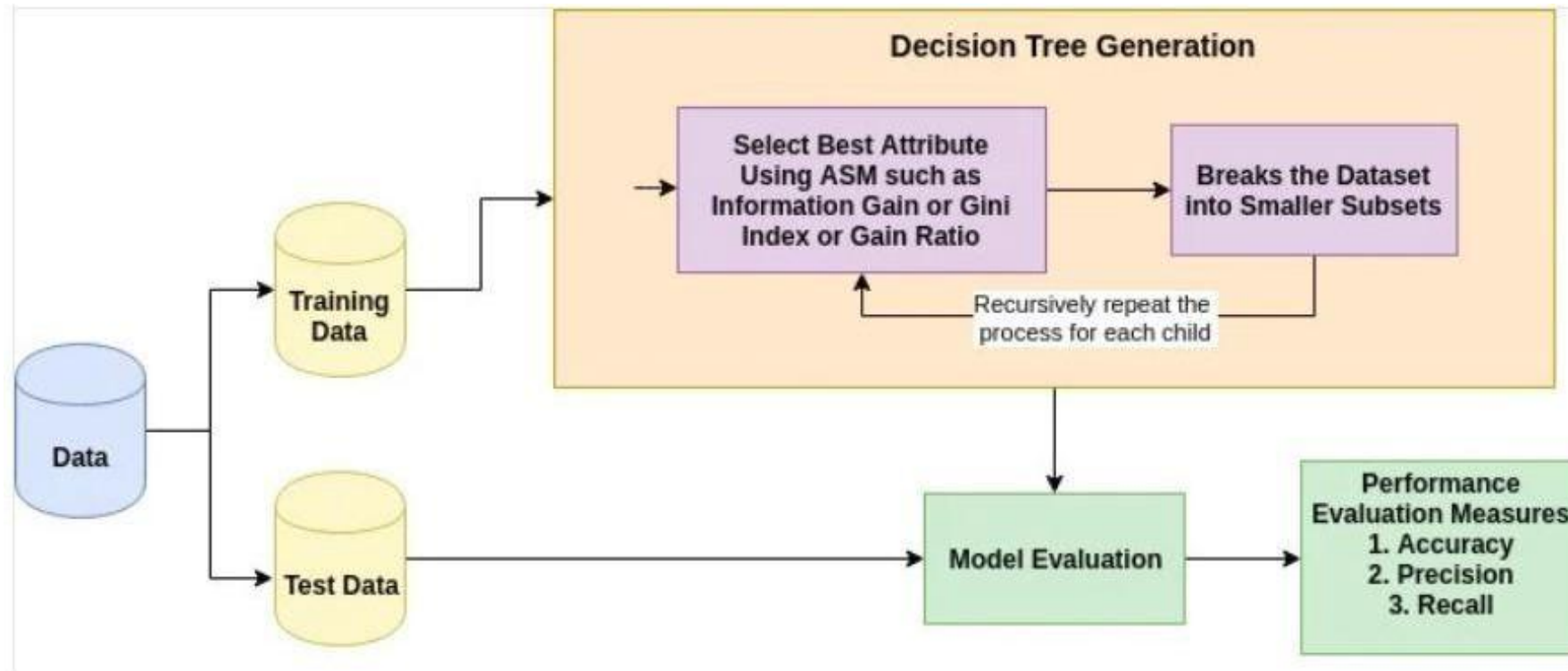
Decision Tree

- A Decision Tree is a supervised learning algorithm used for both classification and regression tasks.
- It works by splitting the data into subsets based on the most significant feature, creating a tree structure where each internal node represents a decision based on a feature, each branch represents the outcome of that decision, and each leaf node represents the final prediction.

RID	age	income	student	credit_rating	Class: buys_computer
1	youth	high	no	fair	no
2	youth	high	no	excellent	no
3	middle_aged	high	no	fair	yes
4	senior	medium	no	fair	yes
5	senior	low	yes	fair	yes
6	senior	low	yes	excellent	no
7	middle_aged	low	yes	excellent	yes
8	youth	medium	no	fair	no
9	youth	low	yes	fair	yes
10	senior	medium	yes	fair	yes
11	youth	medium	yes	excellent	yes
12	middle_aged	medium	no	excellent	yes
13	middle_aged	high	yes	fair	yes
14	senior	medium	no	excellent	no



How does a Decision Tree work?



Training phase: Decision Tree Induction

Decision tree induction refers to the process of generating a decision tree from a given data set.

1. Select the Best Feature to Split:

- Choose the feature that results in the largest gain in information.

2. Split the Dataset:

- Divide the dataset into subsets based on the chosen feature and condition.

3. Repeat for Subsets

- Apply the same process on each subset to create further decision nodes and leaf nodes, until:
 - All data points are correctly classified.
 - A stopping criterion is met (e.g., maximum depth, minimum samples in a leaf).

Training on the sample dataset

- To generate a decision tree from the dataset, the following steps are followed: -
 - Select one of the features (*age*, *income*, *student*, *credit_rating*) to be placed at the root.
 - Based on the feature selected, create 'x' branches from the root, where x – number of unique values of the feature.
 - Follow the same process to place other features at the internal nodes.
- The leaf nodes will have the target classes.

RID	age	income	student	credit_rating	Class: buys_computer
1	youth	high	no	fair	no
2	youth	high	no	excellent	no
3	middle_aged	high	no	fair	yes
4	senior	medium	no	fair	yes
5	senior	low	yes	fair	yes
6	senior	low	yes	excellent	no
7	middle_aged	low	yes	excellent	yes
8	youth	medium	no	fair	no
9	youth	low	yes	fair	yes
10	senior	medium	yes	fair	yes
11	youth	medium	yes	excellent	yes
12	middle_aged	medium	no	excellent	yes
13	middle_aged	high	yes	fair	yes
14	senior	medium	no	excellent	no

Fig. Dataset

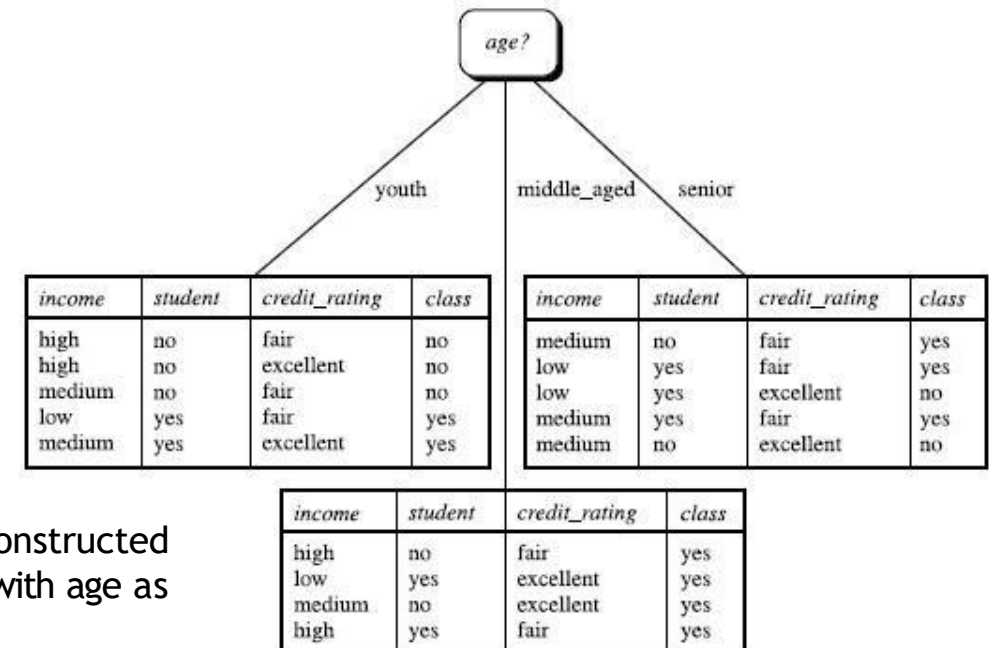


Fig. Partially constructed decision tree with age as the root node

Training phase: Attribute selection measures

- How was age selected to be placed at the root, out of the features (age, income, student, credit_rating)?
 - An attribute selection measure can be used to determine the best attribute.
- Attribute selection measures that can be used: -
 - Information gain
 - Gain ratio
 - Gini index

Information Gain

- Information gain is a metric that measures how much information is gained by splitting a data set on a particular feature.
- Information gain is calculated by comparing the entropy of the original dataset to the entropy of the child sets. A higher information gain indicates that the feature is more effective at splitting the data.
- The entropy of the original dataset (D) is given as follows: -

$$Info(D) = - \sum_{i=1}^m p_i \log_2(p_i), \quad \text{where, } p_i - \text{probability that a tuple in D belongs to class C}$$

- The entropy of the child set is given as follows: -

$$Info_A(D) = \sum_{j=1}^v \frac{|D_j|}{|D|} \times Info(D_j).$$

where, $|D_j|$ – number of samples in subset j
 $|D|$ - total number of samples in the dataset
 $Info(D_j)$ – entropy of subset D_j

- The information gain is given as follows: -

$$Gain(A) = Info(D) - Info_A(D).$$

Information gain calculation

Entropy of dataset

$$Info(D) = -\frac{9}{14} \log_2 \left(\frac{9}{14} \right) - \frac{5}{14} \log_2 \left(\frac{5}{14} \right) = 0.940 \text{ bits.}$$

buys_computer: yes buys_computer: no

Entropy of the child set created when split on the attribute *age*

$$Info_{age}(D) = \frac{5}{14} \times \left(-\frac{2}{5} \log_2 \frac{2}{5} - \frac{3}{5} \log_2 \frac{3}{5} \right) + \frac{4}{14} \times \left(-\frac{4}{4} \log_2 \frac{4}{4} \right) + \frac{5}{14} \times \left(-\frac{3}{5} \log_2 \frac{3}{5} - \frac{2}{5} \log_2 \frac{2}{5} \right)$$

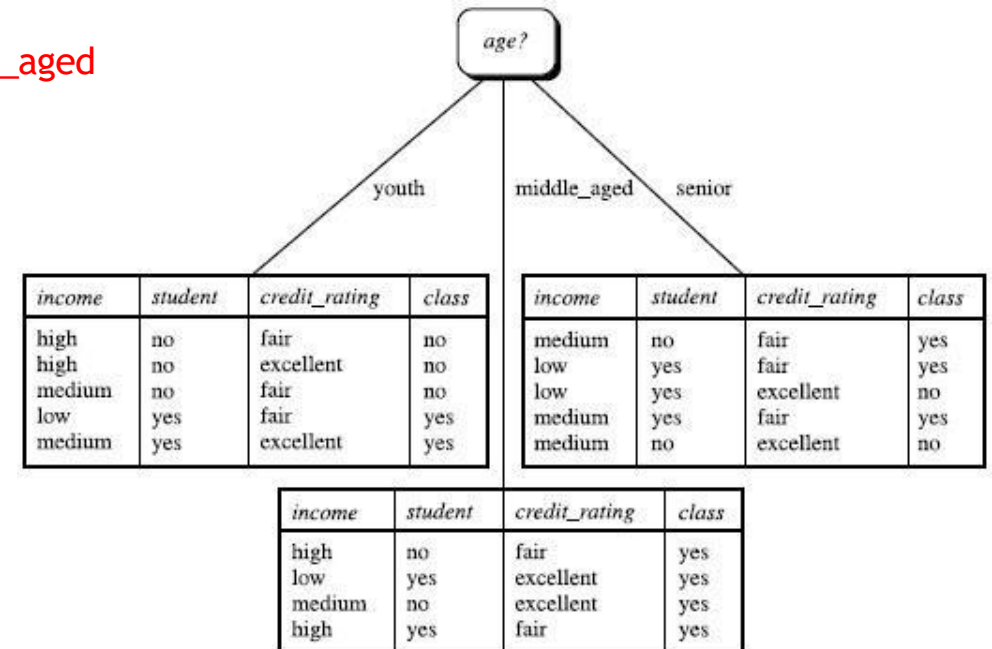
age: youth age: middle_aged age: senior

= 0.694 bits.

Information gain

$$Gain(age) = Info(D) - Info_{age}(D) = 0.940 - 0.694 = 0.246 \text{ bits.}$$

RID	age	income	student	credit_rating	Class: buys_computer
1	youth	high	no	fair	no
2	youth	high	no	excellent	no
3	middle_aged	high	no	fair	yes
4	senior	medium	no	fair	yes
5	senior	low	yes	fair	yes
6	senior	low	yes	excellent	no
7	middle_aged	low	yes	excellent	yes
8	youth	medium	no	fair	no
9	youth	low	yes	fair	yes
10	senior	medium	yes	fair	yes
11	youth	medium	yes	excellent	yes
12	middle_aged	medium	no	excellent	yes
13	middle_aged	high	yes	fair	yes
14	senior	medium	no	excellent	no



Information gain calculation

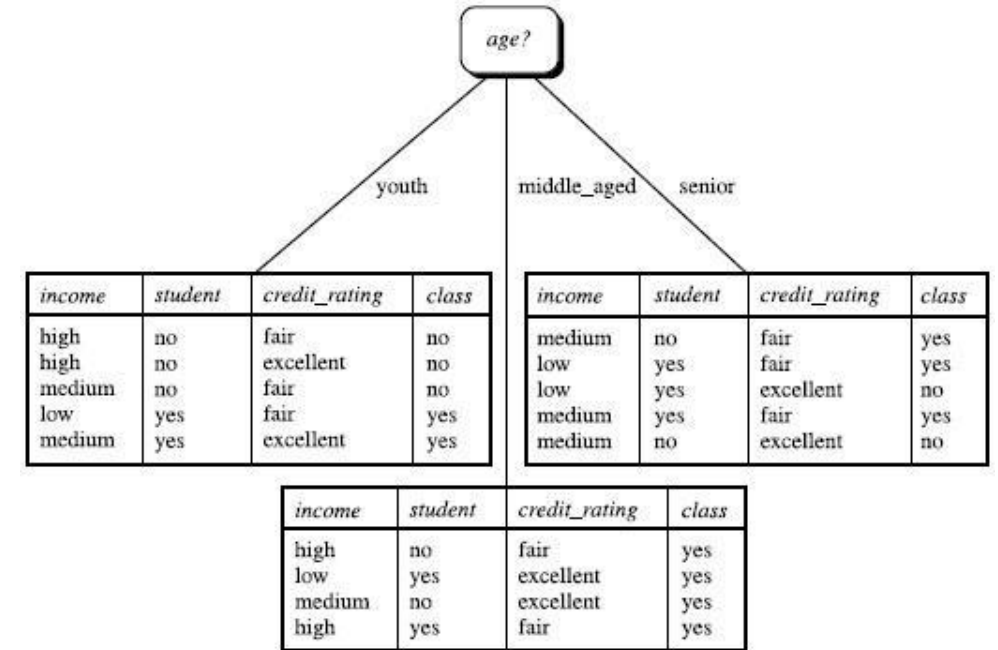
$$\text{Gain}(\text{age}) = \text{Info}(D) - \text{Info}_{\text{age}}(D) = 0.940 - 0.694 = 0.246 \text{ bits.}$$

The gain for the other attributes are

$$\text{Gain}(\text{income}) = 0.029 \text{ bits}$$

$$\text{Gain}(\text{student}) = 0.151 \text{ bits}$$

$$\text{Gain}(\text{credit_rating}) = 0.048 \text{ bits}$$



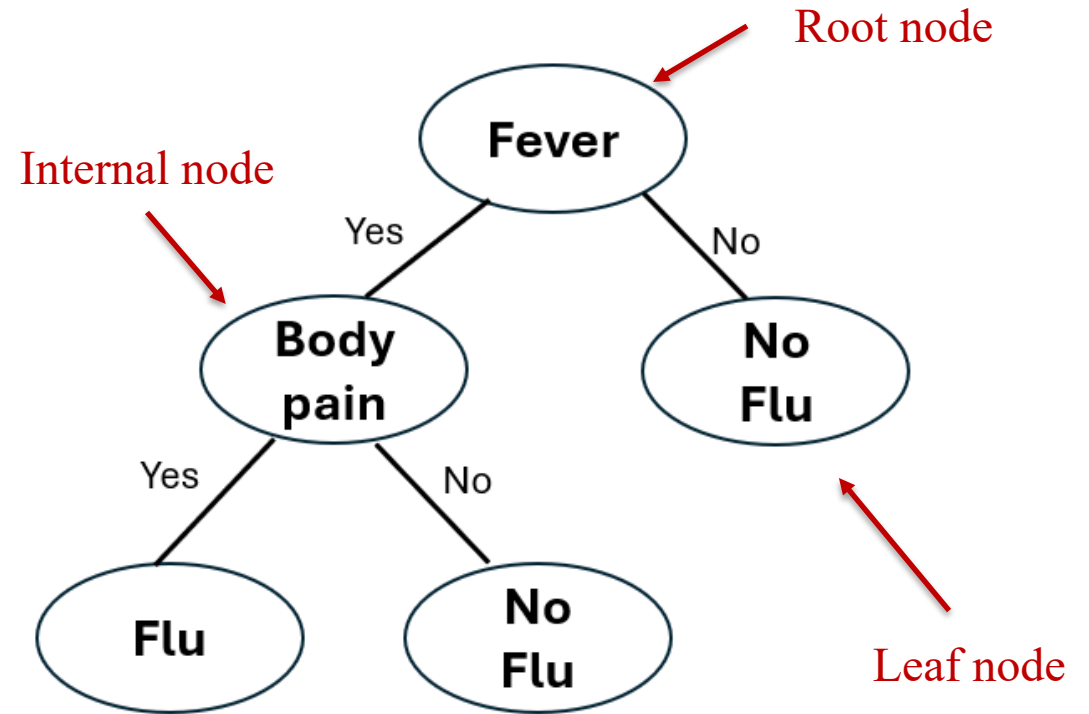
- Since, age has the highest gain value, it is selected to be placed at the root.
- The data is split as shown in the figure. The tuples for the partition *middle_aged* belong to the same category (class: yes), a pure partition. Hence a leaf node is created.
- For the other two impure partitions (*youth*, *senior*), the same procedure is followed on the remaining features (income, student, credit_rating).

Flu prediction - Decision Tree

To predict whether a person has Flu based on basic symptoms.

Fever	Cough	Body Pain	Flu
Yes	Yes	Yes	Yes
Yes	No	Yes	Yes
No	Yes	No	No
No	No	No	No
Yes	Yes	No	No
No	Yes	Yes	No

Dataset



Decision Tree

Decision Tree Induction (Training)

1. Determine which feature $\{Fever, cough, Body\ pain\}$ is to be placed at the root.

How?

Choose each feature and check how well the target classes get separated.

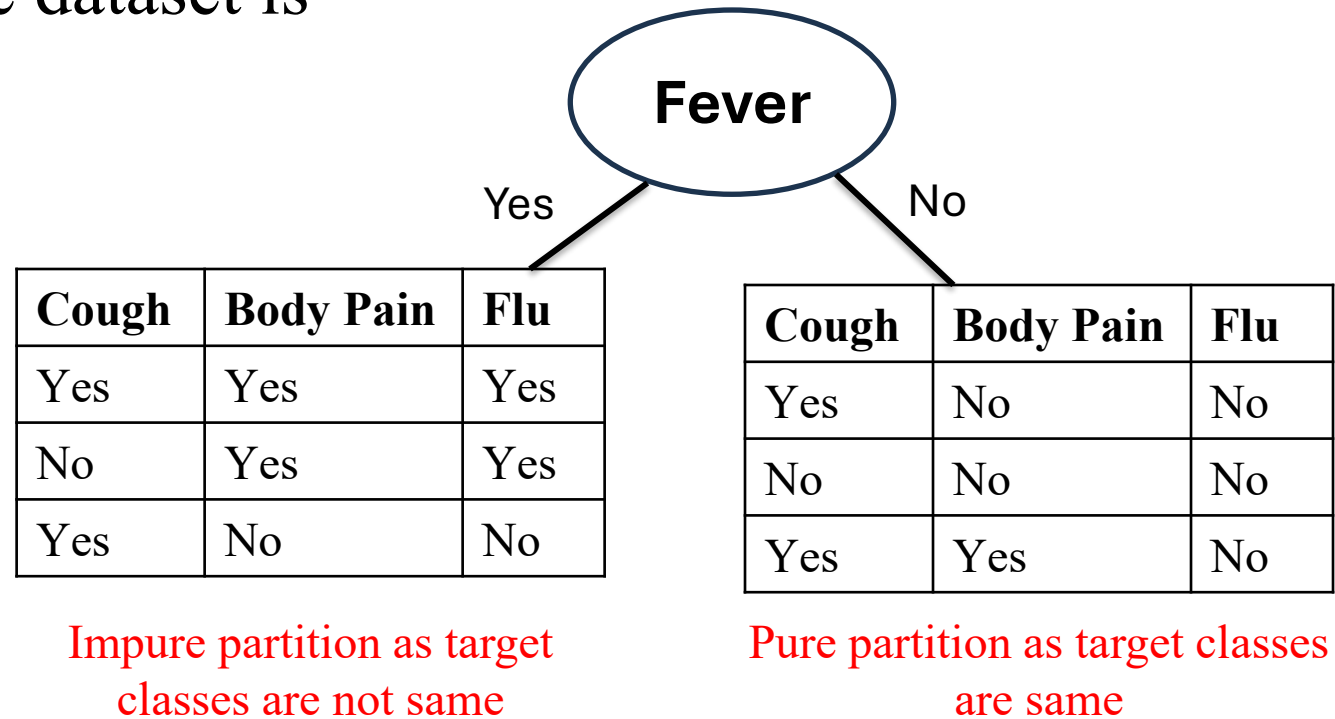
Select the best feature as the root.

Fever	Cough	Body Pain	Flu
Yes	Yes	Yes	Yes
Yes	No	Yes	Yes
No	Yes	No	No
No	No	No	No
Yes	Yes	No	No
No	Yes	Yes	No

Decision Tree Induction (Training)

2. Upon determining the root, the dataset is divided into subsets.

Fever	Cough	Body Pain	Flu
Yes	Yes	Yes	Yes
Yes	No	Yes	Yes
No	Yes	No	No
No	No	No	No
Yes	Yes	No	No
No	Yes	Yes	No



3. Repeat the process for the impure partition.

Decision Tree Induction (Training)

Determining the best feature to be placed at the root.

Compute entropy of the dataset.

$$\text{Entropy}(D) = -\sum p_i \log_2(p_i)$$

where, p_i - probability of class i

Compute entropy of the child sets.

$$\text{Entropy}_F(D) = \sum_{j=1}^v \frac{|D_j|}{|D|} \text{Entropy}(D_j)$$

where, F – selected input feature

$|D_j|$ - no. of samples in subset j

$|D|$ - total no. of samples

$\text{Entropy}(D_j)$ – entropy of subset j

Fever	Cough	Body Pain	Flu
Yes	Yes	Yes	Yes
Yes	No	Yes	Yes
No	Yes	No	No
No	No	No	No
Yes	Yes	No	No
No	Yes	Yes	No

Decision Tree Induction (Training)

Determining the best feature to be placed at the root.
Compute entropy of the dataset.

$$\text{Entropy}(D) = -\sum p_i \log_2(p_i)$$

where, p_i - probability of class I

$$\text{Entropy}(D) = -\left[\frac{2}{6} \log \frac{3}{6} + \frac{4}{6} \log \frac{3}{6} \right] = 0.918$$

Flu = Yes Flu = No

Fever	Cough	Body Pain	Flu
Yes	Yes	Yes	Yes
Yes	No	Yes	Yes
No	Yes	No	No
No	No	No	No
Yes	Yes	No	No
No	Yes	Yes	No

Decision Tree Induction (Training)

Compute entropy of the child set

$$\text{Entropy}_F(D) = \sum_{j=1}^v \frac{|D_j|}{|D|} \text{Entropy}(D_j)$$

$$\text{Entropy}_{\text{Fever}}(D) = \frac{3}{6} \left[\overset{\text{Flu} = \text{Yes}}{-\frac{2}{3} \log \frac{2}{3}} - \overset{\text{Flu} = \text{No}}{\frac{1}{3} \log \frac{1}{3}} \right] \quad \left. \vphantom{\frac{3}{6}} \right\} \text{Fever} = \text{Yes}$$

$$+ \frac{3}{6} \left[\overset{\text{Flu} = \text{Yes}}{-0} - \overset{\text{Flu} = \text{No}}{\frac{3}{3} \log \frac{3}{3}} \right] \quad \left. \vphantom{\frac{3}{6}} \right\} \text{Fever} = \text{No}$$

$$= 0.459$$

Fever	Cough	Body Pain	Flu
Yes	Yes	Yes	Yes
Yes	No	Yes	Yes
No	Yes	No	No
No	No	No	No
Yes	Yes	No	No
No	Yes	Yes	No



Cough	Body Pain	Flu
Yes	Yes	Yes
No	Yes	Yes
Yes	No	No



Cough	Body Pain	Flu
Yes	No	No
No	No	No
Yes	Yes	No

Decision Tree Induction (Training)

Compute information gain

$$\text{Info. Gain (Fever)} = \text{Entropy}(D) - \text{Entropy}_{\text{Fever}}(D) \\ = 0.459$$

$$\text{Info. Gain (Cough)} = \text{Entropy}(D) - \text{Entropy}_{\text{Cough}}(D) \\ = 0.044$$

$$\text{Info. Gain (Body Pain)} = \text{Entropy}(D) - \text{Entropy}_{\text{Body Pain}}(D) \\ = 0.458$$

Choose the feature with the highest Info. Gain as the root.

Fever	Cough	Body Pain	Flu
Yes	Yes	Yes	Yes
Yes	No	Yes	Yes
No	Yes	No	No
No	No	No	No
Yes	Yes	No	No
No	Yes	Yes	No

Spam Email Classification - Demo

Step1: Importing the libraries

```
import pandas as pd  
from sklearn.feature_extraction.text import CountVectorizer  
from sklearn.model_selection import train_test_split  
from sklearn.tree import DecisionTreeClassifier  
from sklearn.metrics import accuracy_score, confusion_matrix,  
classification_report  
from sklearn.tree import plot_tree  
import matplotlib.pyplot as plt
```

Step 1: Dataset Preparation

A labeled dataset of email messages is prepared.

Each message is tagged as spam or ham (normal email).

The dataset contains two columns

text → email content

label → class (spam / ham)

Email	label
Hi we will meet tomorrow at the office	ham
Congratulations you have won a free iPhone click now	spam
Please find the attached assignment for today	ham
Limited time offer buy one get one free	spam

The dataset is imported into a pandas dataframe (df).

Step 2: Data preprocessing (Label Encoding)

Textual labels are represented using numbers.

ham $\rightarrow$ 0

spam $\rightarrow$ 1

```
df["label"] = df["label"].map({"ham": 0, "spam": 1})
```

Email	label
Hi we will meet tomorrow at the office	0
Congratulations you have won a free iPhone click now	1
Please find the attached assignment for today	0
Limited time offer buy one get one free	1

Step 3: Text Feature Extraction (Bag of Words)

Technique Used: CountVectorizer

It is a feature extraction technique that converts textual data into numerical form by counting how often words appear in a document.

Step 1: Tokenization of the email text into words

Example: “*Win a free prize*”  [“*Win*”, “*a*”, “*free*”, “*prize*”]

Step 2: Removal of common English stop-words (*is, am, are, the, a, an, in*)

[“*Win*”, “*a*”, “*free*”, “*prize*”]  [“*Win*”, “*free*”, “*prize*”]

Step 3: Text Feature Extraction (Bag of Words)

Step 3: Building a vocabulary of unique words

Collect all unique words and assign each word a feature index

Example

Email 1: "free win prize"

Email 2: "meeting tomorrow"

Feature vector: ["free", "win", "prize", "meeting", "tomorrow"]

Convert each email into a word-count vector

Email	free	win	prize	meeting	tomorrow
"free win prize"	1	1	1	0	0
"meeting tomorrow"	0	0	0	1	1

Bag of Words representation, word order does not matter, frequency matters

Step 4: Feature Matrix and Target Vector Formation

Dataset is represented as (X, Y)

Feature Matrix (X)

Index	free	win	prize	meeting	tomorrow
0	1	1	1	0	0
1	0	0	0	1	1

Target Label (Y): $y - \{\text{ham}, \text{spam}\} \rightarrow \{0, 1\}$

Index	free	win	prize	meeting	tomorrow	Label
0	1	1	1	0	0	1
1	0	0	0	1	1	0

Steps 3 and 4

```
vectorizer = CountVectorizer(stop_words="english")  
X = vectorizer.fit_transform(df["text"])  
y = df["label"]
```

Step 5: Train–Test Split

Dataset is divided into training set and testing set.

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42
)
```

Step 6: Decision Tree is trained

```
dt = DecisionTreeClassifier(
    criterion="entropy", # uses Information Gain
    max_depth=5,         # prevents overfitting
    random_state=42
)
dt.fit(X_train, y_train)
```

Step 6: Decision Tree is tested

y_pred = dt.predict(X_test)

Step 9: Prediction done using the decision tree

sample = ["You have won a free cash prize"]

Output - spam

K Nearest Neighbor (KNN)

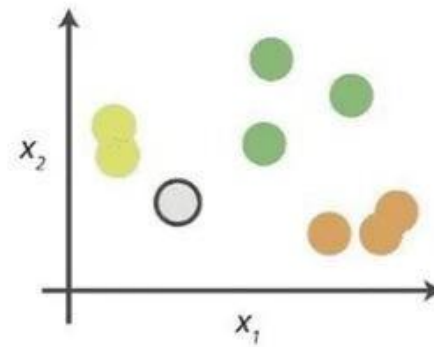
- The K Nearest Neighbor (KNN) is an algorithm that can be used for both classification and regression (mostly used for classification).
- It relies on the idea that similar data points tend to have similar labels or values.
- Training phase: stores the entire dataset (lazy learner)
- Testing phase: calculates distance between input data point to all the training examples using a distance metric (E.g. Euclidean dist., Manhattan dist., etc.)
- Parameter tuning: value of 'k' determined.
- Classify input value using the value of 'k'.



KNN

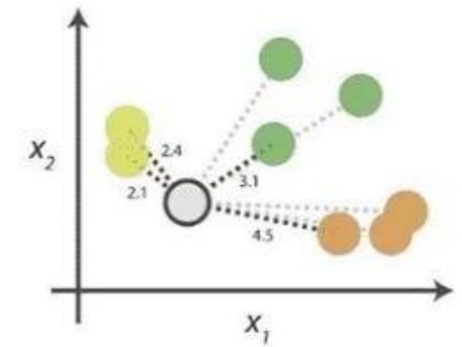
1. **Choose K** → the number of neighbours to consider.
2. **Compute distances** → for a new input, calculate its distance (e.g., Euclidean, Manhattan) to all training samples.
3. **Find the K nearest neighbors** → pick the K training points closest to the input.
4. **Make a decision** → assign the most frequent class among the K neighbors.

0. Look at the data



Say you want to classify the grey point into a class. Here, there are three potential classes - lime green, green and orange.

1. Calculate distances



Start by calculating the distances between the grey point and all other points.

2. Find neighbours

Point Distance			
		2.1	→ 1st NN
		2.4	→ 2nd NN
		3.1	→ 3rd NN
		4.5	→ 4th NN

Next, find the nearest neighbours by ranking points by increasing distance. The nearest neighbours (NNs) of the grey point are the ones closest in dataspace.

3. Vote on labels

Class	# of votes	
	2	➔ Class wins the vote! Point is therefore predicted to be of class .
	1	
	1	

Vote on the predicted class labels based on the classes of the k nearest neighbours. Here, the labels were predicted based on the k=3 nearest neighbours.

KNN - Example

- There are two categories A and B, and we have a new data point to classify.

- Let $K = 5$
- Calculate distance (Euclidean distance) of new data point to all data points.

$$Dist(p_1, p_2) = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

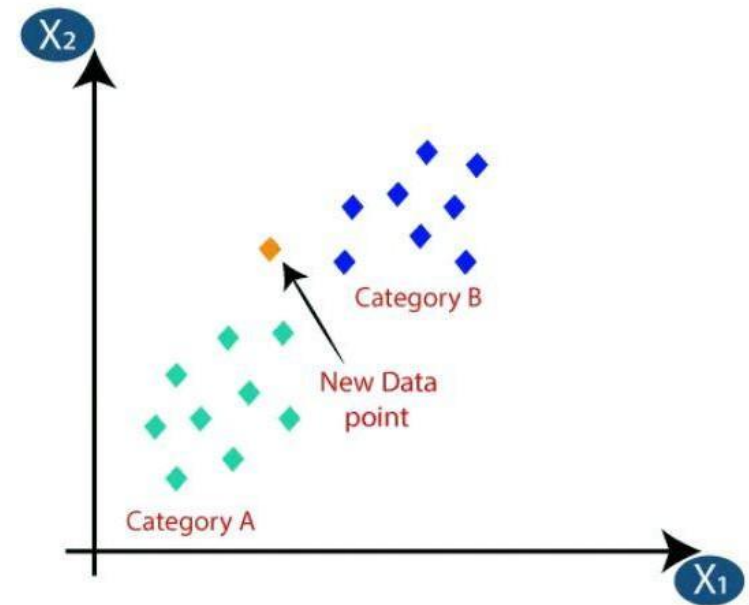
- Take K nearest neighbors as per the calculated distance.

Nearest neighbors in category A = 3

Nearest neighbors in category B = 2

- The new data point is assigned to category A

*K nearest neighbours
encircled*



Distance of new data point to every data point calculated.



Determining value of 'K'

- The selection of the optimal 'k' value depends on the specific dataset and the characteristics of the problem, and it can significantly impact the performance of the algorithm.
- Considerations
 - Odd value of 'k' more suitable.
 - Smaller value of 'k' can be noisy, while larger can have bias and is more computationally expensive.
 - Cross validation
 - $K = \sqrt{N}$, where N = no. of samples in the dataset.

Example: Determine the 'Default' status for Andrew.

Customer	Age	Loan	Default
John	25	40000	N
Smith	35	60000	N
Alex	45	80000	N
Jade	20	20000	N
Kate	35	120000	N
Mark	52	18000	N
Anil	23	95000	Y
Pat	40	62000	Y
George	60	100000	Y
Jim	48	220000	Y
Jack	33	150000	Y
Andrew	48	142000	?

Training
data

Customer	Age	Loan	Default	Euclidean distance
John	25	40000	N	1,02,000.00
Smith	35	60000	N	82,000.00
Alex	45	80000	N	62,000.00
Jade	20	20000	N	1,22,000.00
Kate	35	120000	N	22,000.00
Mark	52	18000	N	1,24,000.00
Anil	23	95000	Y	47,000.01
Pat	40	62000	Y	80,000.00
George	60	100000	Y	42,000.00
Jim	48	220000	Y	78,000.00
Jack	33	150000	Y	8,000.01
Andrew	48	142000	?	

Step 1: Calculate the distance to all data
points

Example: Determine the 'Default' status for Andrew.

Customer	Age	Loan	Default	Euclidean distance	Minimum Euclidean Distance
John	25	40000	N	1,02,000.00	
Smith	35	60000	N	82,000.00	
Alex	45	80000	N	62,000.00	5
Jade	20	20000	N	1,22,000.00	
Kate	35	120000	N	22,000.00	2
Mark	52	18000	N	1,24,000.00	
Anil	23	95000	Y	47,000.01	4
Pat	40	62000	Y	80,000.00	
George	60	100000	Y	42,000.00	3
Jim	48	220000	Y	78,000.00	
Jack	33	150000	Y	8,000.01	1
Andrew	48	142000	?		

Step 2: Assume $k = 5$

Step 3: Find five minimum distance values.

Out of 5 chosen records there are three Ys and two Ns.

Andrew's default status is taken as Y.